

Bash and Terminal

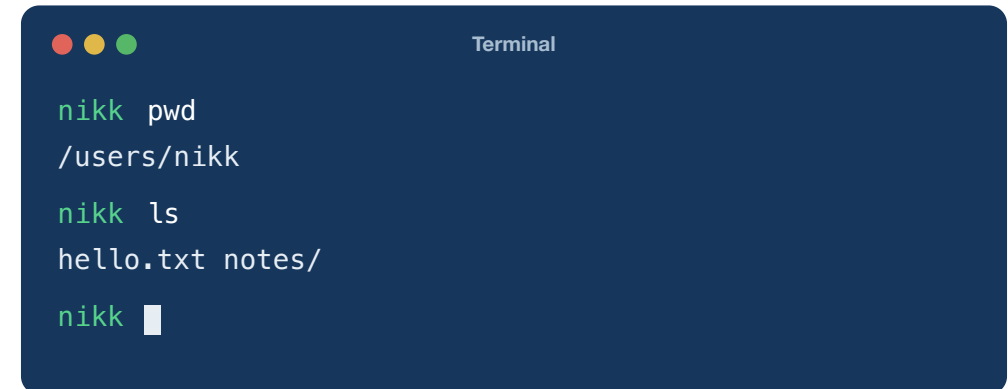
Full Stack: Frontend, Backend, Database & Deployment · Class 3

GUI vs. Command Line

- Two ways to tell a computer what to do: point-and-click (GUI) or type commands (CLI)
- A GUI hides steps behind menus, icons, and windows
- The command line skips the menus and acts directly
- Doing one thing is fine either way — repeating it 60 times is where they differ

What is a Terminal?

- The window/application you type commands into
- Every command you run, and its output, appears right below it — like a log
- It's not magic — hackers, programmers, and engineers all just type text into this



```
Terminal  
nikk pwd  
/users/nikk  
nikk ls  
hello.txt notes/  
nikk █
```

`pwd` — Where Am I?

- `pwd` = Print Working Directory
- Shows your current location in the file system, as a path
- GUI equivalent: the address bar / breadcrumb path in Finder or Explorer

`ls` — What's Here?

- ls = List
- Shows the files and folders inside your current directory
- GUI equivalent: the contents you see when a folder window opens

`mkdir` — Make a Directory

- `mkdir <name>` creates a new folder
- Same result as right-click → New Folder in the GUI
- One command per folder means naming 60 folders is 60 fast, exact commands — not 60 careful clicks

cd — Change Directory

- `cd <folder>` moves you inside a folder
- `cd ..` moves you up one level, back out
- GUI equivalent: double-clicking into a folder, or clicking "back"

touch — Create a File

- `touch <filename>` creates a new, empty file instantly
- Its real job is updating a file's timestamp — creating it is just what happens when the file doesn't exist yet
- GUI equivalent: right-click → New File

cat — Read a File

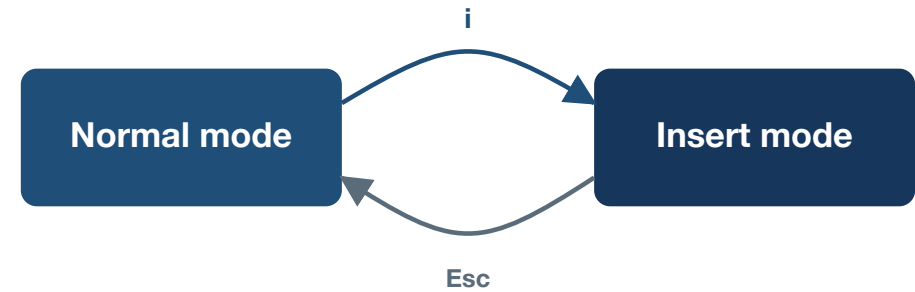
- `cat <filename>` prints a file's contents straight to the terminal
- `cat` = concatenate — its real job is joining files together; printing one is the simplest case
- A fast way to peek at a file without opening an editor

Vim: A Different Kind of Editor

- `vi <filename>` (or `vim`) opens the file inside a full-screen editor, right in the terminal
- Typing doesn't insert text right away — that surprises everyone at first
- That's because Vim has "modes" — the same key does different things depending on the mode

Vim's Modes

- Normal mode: keys are commands, not text — this is where Vim opens
- Insert mode: keys type text, like a normal editor
- Press `i` to enter Insert mode; press `Esc` to return to Normal mode



Saving & Quitting Vim

- Vim commands are typed in Normal mode, starting with `:`
- `:wq` — write (save) and quit
- `:q!` — quit without saving, discarding changes
- Feeling "stuck" in Vim almost always just means being in the wrong mode

cp — Copy a File

- `cp <source> <destination>` creates a duplicate
- The original file is untouched — now there are two copies

mv — Move or Rename

- `mv <source> <destination>` relocates a file
- Renaming is the same operation — just give a new name in the same folder as the destination
- Moving and renaming are, underneath, one and the same command

rm — Delete a File

- `rm <file>` deletes a file permanently
- Unlike the GUI's Trash/Recycle Bin, there is usually no undo
- Always double-check before running `rm` — it is risky by design

`rm -r` and `rm -ri` — Deleting Directories

- Plain `rm` only deletes files — it refuses on a directory
- `rm -r` deletes a directory and everything inside it, recursively
- `rm -ri` does the same, but asks for confirmation before deleting each item
- Still no undo — `-r` just means it can now do more damage per command

Terminal vs. Shell

- Terminal: the window/app you see and type into — just the interface
- Shell: the program that actually reads your text and figures out what to do
- The terminal displays; the shell interprets and executes

Bash & Other Shells

- Bash is one specific shell program — the most common one
- Other shells exist too: Zsh, Fish, and more
- All of them share the same core commands: pwd, ls, cd, mkdir, cp, mv, rm...
- Switching shells rarely breaks these basics — differences mostly show up in advanced scripting

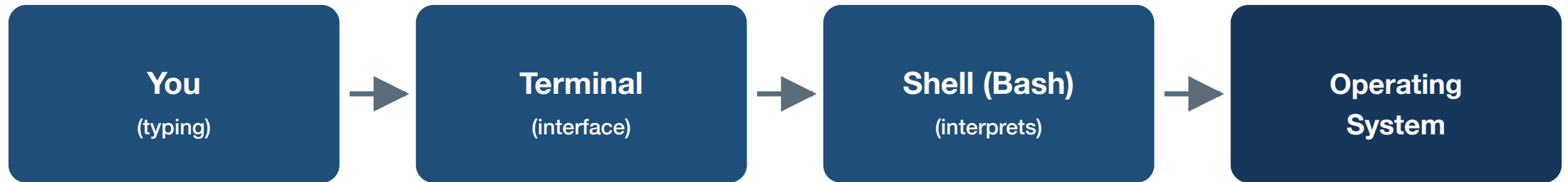
`echo $0` — Which Shell Am I In?

- `echo` prints text — or a variable's value — to the terminal
- `$0` holds the name of the shell currently running
- `echo $0` prints something like `/bin/bash` or `/bin/zsh`
- A quick way to check which shell is actually interpreting your commands

PS1 — Customizing Your Prompt

- PS1 is the shell variable that controls what your prompt looks like
- That's where "nikk" (or "user@laptop") before your cursor comes from
- PS1="something" changes the prompt live
- Each shell keeps its own PS1 — it's not something the terminal decides

The Full Chain



each layer hands your command down to the next, and the answer travels back up

Redirecting Output: `>` vs `>>`

- `echo "text"` normally just prints to the terminal
- `>` redirects that output into a file instead — overwriting whatever was there
- `>>` redirects too, but appends to the end instead
- `echo "This is a new line" >> my_file.txt` keeps adding lines without erasing existing content

Getting Bash on Windows

- Windows doesn't ship with Bash by default
- Option 1: WSL (Windows Subsystem for Linux) — a full Linux environment inside Windows
- Option 2: Git Bash, from gitforwindows.org — a lightweight Bash emulator bundled with Git
- Either gives you the same commands covered today